

What is Operator Overloading?

👉 Operator Overloading means:

Using the same operator for different purposes depending on the data

Simple Understanding

```
int a = 10 + 20;    // addition
String s = "Hi " + "Java"; // concatenation
```

👉 Here + is used for:

- Numbers → Addition
- Strings → Joining text

Does Java Support Operator Overloading?

✗ Java does NOT allow you to create your own operator behavior

👉 You cannot change how operators work

What Java Allows

- Predefined operator behavior
- + can work with:
 - Numbers → Addition
 - Strings → Concatenation

Special Case: String Concatenation

```
String result = "Hello " + "World";  
System.out.println(result);
```

👉 Output:

Hello World

Internal Working (Simple Idea)

Java converts:

"Hello " + "World"

Into:

```
StringBuilder sb = new StringBuilder();  
sb.append("Hello ");  
sb.append("World");  
sb.toString();
```

Important Behavior of +

Case 1: Left to Right Execution

```
System.out.println(10 + 20 + "Java");  
System.out.println("Java" + 10 + 20);
```

👉 Output:

30Java
Java1020

Case 2: Mathematical Priority

```
System.out.println("Result: " + 10 * 2);
```

👉 Output:

Result: 20

✓ Multiplication happens first

Case 3: Mixing Types

```
System.out.println("Age: " + 25);
```

👉 Output:

Age: 25

WHAT-IF SCENARIOS

Case 1: Using + with String and number

```
System.out.println("Marks: " + 90);
```

✓ Output:

Marks: 90

Case 2: Wrong expectation of addition

```
System.out.println("10" + 20);
```

✓ Output:

1020

👉 Reason: String concatenation happens

Case 3: Arithmetic first, then concatenation

```
System.out.println("Total: " + (10 + 20));
```

✓ Output:

Total: 30